

Heads-Up Limit Hold'em: A Monte-Carlo Equity / Expected-Value Agent

Yugank Anand Singh 251221
IIT Kanpur

Report

Abstract

This report documents the design, mathematical justification, and empirical evaluation of **CustomPokerBot**, an autonomous agent for Heads-Up Limit Texas Hold'em (HULHE) built entirely from scratch in Python, without any external poker libraries. The system consists of (i) an object-oriented card and hand-evaluation engine, (ii) a from-scratch game engine that implements the fixed-limit betting structure, blind posting, and cap rule specified for this tournament, and (iii) a decision policy that estimates hand equity via Monte Carlo rollouts and converts that equity into an action using a pot-odds/expected-value framework. We describe the state representation, the statistical justification for the equity estimator, the EV derivation for each of the three legal actions, and report win-rate/bb-per-100 results against three baseline opponents of increasing sophistication. We close with an honest discussion of the model's limitations – principally, that it assumes a uniformly random opponent range rather than a history-conditioned one – and outline how the architecture extends toward Counterfactual Regret Minimization (CFR).

Contents

1	Problem Setting	2
2	System Architecture	2
3	Hand Strength Quantization	2
3.1	Monte Carlo Estimator	2
3.2	Why Monte Carlo Instead of an Exact Enumeration	3
4	Decision Policy	3
4.1	Explicit Modeling Choices and Their Justification	4
5	Complexity and Runtime	4
6	Empirical Results	4
7	Limitations and Future Work	5
8	Conclusion	5

1 Problem Setting

Heads-Up Limit Hold'em is a two-player, imperfect-information, zero-sum game. Unlike the cart-pole environment used earlier in this course, the agent does not observe the full state: an opponent's two hole cards are hidden until (and unless) a showdown occurs. Formally, at any decision node the agent observes a public information set

$$s = (h, c, \text{pot}, \text{stack}, \text{to_call}, \mathcal{A})$$

where h is our two hole cards, $c \in \{0, 3, 4, 5\}$ community cards (pre-flop, flop, turn, river), pot and stack are chip counts, to_call is the chips required to continue, and $\mathcal{A} \subseteq \{\text{FOLD}, \text{CALL}, \text{RAISE}\}$ is the legal action set enforced by the arena (which already encodes the fixed-limit betting increment and the four-bet cap rule for us). The opponent's private information o (their hole cards) is unobserved. Our task is to choose $a \in \mathcal{A}$ to maximize expected chip return under uncertainty over o and over the remaining, not-yet-dealt community cards.

2 System Architecture

The submission consists of three single-responsibility modules, as recommended by the assignment brief:

- `card.py` – an immutable `Card` class implementing `__eq__`, `__lt__` (via `functools.total_ordering`), and `__hash__`, plus a cached factory (`Card.get`) so that Monte Carlo rollouts reuse the 52 possible card objects instead of re-validating a string millions of times.
- `hand_evaluator.py` – given 5–7 cards, returns the best 5-card hand as a totally-ordered score tuple (category, tiebreak₁, ...), using `itertools.combinations` to enumerate the $\binom{7}{5} = 21$ candidate hands and a rank-count array (in place of `collections.Counter`, for speed – see Section 5) to detect pairs/trips/quads/flushes.
- `agent.py` – the required entry point, containing `BasePokerBot` (unchanged) and `CustomPokerBot`, described in Section 4. All deck handling, equity sampling, and the EV-based decision policy live here.

3 Hand Strength Quantization

Rather than hand-crafting a lookup table of starting-hand strengths (which the rules explicitly disallow, and which would not generalize past the pre-flop street anyway), we quantize hand strength as a single scalar, *equity*: the probability of winning the pot if the hand were taken to showdown right now against a uniformly random opponent hand, with any undealt community cards also drawn uniformly at random.

3.1 Monte Carlo Estimator

Let $\mathcal{D}$ be the 52-card deck minus the cards we can already see (our 2 hole cards plus $|c|$ community cards). Let $k = 5 - |c|$ be the number of community cards still to come. On each of N independent trials $i = 1, \dots, N$:

1. Draw $2 + k$ cards without replacement from $\mathcal{D}$: two for a hypothetical opponent hand o_i , and k to complete the board, c_i^+ .

2. Evaluate the best 5-of-7 hand for us $(h, c \cup c_i^+)$ and for the opponent $(o_i, c \cup c_i^+)$ using `hand_evaluator`.
3. Record $x_i = 1$ if we win, $x_i = 0.5$ on a tie, $x_i = 0$ if we lose.

The equity estimate is the sample mean

$$\hat{p} = \frac{1}{N} \sum_{i=1}^N x_i.$$

Since each $x_i \in \{0, 0.5, 1\}$ is an i.i.d. draw with mean equal to the true equity p and bounded support, Hoeffding’s inequality gives a distribution-free confidence bound

$$\Pr(|\hat{p} - p| \geq \varepsilon) \leq 2 \exp(-2N\varepsilon^2).$$

For example, with $N = 250$ (our river sample count) and $\varepsilon = 0.05$, this bound is $2 \exp(-2 \cdot 250 \cdot 0.0025) \approx 0.29$ – a loose worst-case bound, but the empirical standard error $\sigma/\sqrt{N} \approx 0.5/\sqrt{250} \approx 0.032$ is small enough for the coarse-grained (FOLD/CALL/RAISE) decision this feeds into. We use fewer samples pre-flop ($N = 150$) and progressively more as the board fills in ($N = 150/200/250$ for flop/turn/river) – earlier streets have more future betting rounds to average out estimator noise, while later streets are higher-stakes single decisions where precision matters more and there is no more hidden information to reveal, so extra samples are pure variance reduction rather than wasted compute on an estimate that will be superseded next street anyway.

3.2 Why Monte Carlo Instead of an Exact Enumeration

An exact pre-flop equity computation would require enumerating all $\binom{50}{2} = 1225$ opponent hands (post-flop, all opponent hands times all remaining board runouts) and is tractable pre-flop, but becomes expensive on the flop/turn ($\binom{47}{2} \times \binom{45}{2}$ board run-outs, etc.) if repeated at every one of thousands of arena hands under a wall-clock budget. Monte Carlo sampling trades a small, well-understood amount of statistical noise for a sample complexity that is *constant* in the number of unseen cards, which is what makes real-time play across a 10,000-hand match feasible.

4 Decision Policy

Given the equity estimate $\hat{p}$, we approximate the one-shot expected value of each legal action under the pot-odds model:

$$\text{EV}(\text{FOLD}) = 0 \tag{1}$$

$$\text{EV}(\text{CALL}) = \hat{p} \cdot (\text{pot} + \text{to_call}) - \text{to_call} \tag{2}$$

$$\text{EV}(\text{RAISE}) = p_f \cdot \text{pot} + (1 - p_f) \left[\hat{p} \cdot (\text{pot} + \text{to_call} + 2b) - (\text{to_call} + b) \right] \tag{3}$$

where b is the current street’s fixed bet increment (2 chips pre-flop/flop, 4 chips turn/river, matching the arena’s fixed-limit structure) and p_f is a conservative, street-dependent *fold equity* constant representing how often we expect a raise to make the opponent fold immediately (we use $p_f \in \{0.12, 0.18, 0.20, 0.22\}$ for pre-flop/flop/turn/river respectively – deliberately low, since fixed-limit opponents – especially calling-station-style ones – fold far less often than in no-limit play, so overestimating p_f would make the bot bluff too liberally).

Derivation of EV(CALL). If we call, we invest `to_call` additional chips into a pot that will total `pot + to_call`. With probability $\hat{p}$ we win that whole pot back; the chips we personally contributed are, from an EV-going-forward standpoint, already staked, so the marginal value of calling is $\hat{p} \cdot (\text{pot} + \text{to_call})$ against a cost of `to_call`.

Derivation of EV(RAISE). A raise adds `to_call + b` to our own investment and b to the final pot size if called. Two branches occur: with probability p_f the opponent folds and we win the current pot immediately (a "clean" gain of `pot`, since further chips never go in); with probability $1 - p_f$ the opponent calls, and we reach the same kind of showdown-EV expression as the CALL case, but against the larger raised pot (`pot + to_call + 2b`) and larger personal cost (`to_call + b`).

We then select

$$a^* = \arg \max_{a \in \mathcal{A}} \text{EV}(a),$$

breaking near-ties (within 10^{-9}) in favor of the least committal legal action (`FOLD > CALL > RAISE` in tie-break priority) to control variance across a long match, since a Monte-Carlo-noisy EV difference of a few hundredths of a chip should not swing a real decision.

4.1 Explicit Modeling Choices and Their Justification

- **Uniform opponent range.** We assume the opponent's hole cards are uniformly distributed over all cards we cannot see. This is the maximum-entropy assumption given no observed opponent actions from earlier in the hand (the interface does not expose betting history to `get_action`), and is a standard, defensible starting point – see Section 7 for its main weakness.
- **No explicit look-ahead.** `EV(RAISE)` only models one additional street of betting (the immediate call/fold response), not the full remaining game tree. A true solution would require solving a subgame via CFR or another equilibrium-finding method; we chose the cheaper one-ply approximation to guarantee the bot finishes 10,000 hands within a reasonable wall-clock budget (see Section 5).
- **Fixed, hand-independent fold equity p_f .** A more sophisticated bot would condition p_f on board texture and opponent tendencies observed so far in the match; we use a single conservative constant per street as a first-order approximation.

5 Complexity and Runtime

Each Monte Carlo trial evaluates two 7-card hands, each of which brute-forces $\binom{7}{5} = 21$ five-card hands, so a single equity estimate with N samples costs $O(42N)$ hand evaluations. Profiling showed that replacing `collections.Counter` with a fixed-size (15-entry) rank-count array inside `evaluate_5` gave roughly a 4–5 $\times$ speedup (from $\approx 0.16\text{s}$ to $\approx 0.02\text{s}$ per decision at our sample sizes), and caching the 52 possible `Card` objects (rather than re-constructing/re-validating one from a string on every sample) removed further avoidable overhead. With this optimization, a full HULHE hand (3–4 betting decisions on average) takes roughly 50–90ms end to end in local self-play, which comfortably fits the multi-thousand-hand, round-robin tournament format described in the assignment.

6 Empirical Results

`CustomPokerBot` was played head-to-head against three baseline opponents for 500 hands each, alternating the dealer button every hand:

Opponent	Win rate (ours)	Net chips (ours)	bb/100
RandomBot	92.6%	+2921	+292.1
CallingStationBot	45.8%	+146	+14.6
TightAggressiveBot	66.8%	-705	-70.5

Table 1: 500-hand local benchmark, dealer button alternated each hand, big blind = 2 chips.

The bot dominates a uniformly random opponent (as expected – equity-based play beats no strategy at all by a wide margin) and shows a small but positive edge against a bot that never folds (a calling station is, by construction, easy to value-bet against but hard to bluff, which matches our conservative fold-equity assumption). Against the tight-aggressive baseline, however, our bot has a *higher raw win rate* (66.8% of hands won) but a *negative* chip result – it wins many small pots and loses fewer, but larger, ones. This is diagnostic, not surprising, and is discussed next.

7 Limitations and Future Work

The single biggest weakness exposed by Section 6 is the **uniform-range assumption**. TightAggressiveBot only continues in a hand with genuinely strong holdings (paired preflop, or a rank matching its hole cards, or a strong preflop combination); once it has voluntarily put in a raise, its *true* range is much stronger than “any two cards.” Our equity estimator does not condition on the fact that the opponent chose to raise, so it systematically *overestimates* our equity in exactly the spots where the opponent has shown strength – leading us to continue (call/raise) with hands that are, conditionally, worse than our unconditional Monte Carlo estimate suggests. This is the classic gap between “unconditional hand strength” and “range-aware hand strength” in poker AI.

Two extensions would directly address this, and are natural next steps:

1. **Opponent range narrowing.** Track observed betting aggression within a hand (available implicitly via jumps in `amount_to_call` across successive calls to `get_action`) and bias the Monte Carlo opponent-hand sampler toward a plausible “raising range” (e.g., pairs and high-card combinations) rather than sampling uniformly, using rejection sampling over `itertools.combinations` of the remaining deck.
2. **Counterfactual Regret Minimization (CFR).** Because HULHE with a small, discretized action set (FOLD/CALL/RAISE, capped bet sizes) has a comparatively small game tree, a tabular or abstracted CFR solver is feasible to train offline (using only `numpy`, as permitted) and would converge toward a Nash-equilibrium strategy that is unexploitable by construction, rather than relying on hand-tuned fold-equity constants.

8 Conclusion

We implemented a complete, dependency-free HULHE stack: card representation, hand evaluation, a rules-accurate fixed-limit game engine, and a Monte-Carlo-equity-driven EV agent. The agent convincingly beats unsophisticated opponents and holds a small edge over a “calls everything” strategy, while a tight, disciplined opponent exposes the expected weakness of an unconditional-equity model – a result we view as a successful stress test of the architecture rather than a failure, since it points precisely at the well-understood next step (range-aware or CFR-based play) rather than an unexplained gap.